## Informacje ogólne

1. **Nazwa projektu:** Undertale Demo
2. **Silnik:** Unity
3. **Język:** C#
4. **Opis:** Jako Zwykła osoba wpadasz do nieznanego ci uniwersum. W trakcie twojej przygody napotkasz wiele stworzeń z którymi możesz wejść w interakcję. Będą cię atakować i to od ciebie zależy co postanowisz z nimi zrobić. Porozmawiasz z nimi? a może wyciągniesz broń i odpowiesz ogniem na ogień. Pamiętaj, każdy wybór ma swoje konsekwencje.

## Struktura projektu

```
Assets/
│└─ Code/
│      ├─ Inventory/
│      │      └─ Inventory.cs
│      │      └─ InventoryManagerUI.cs
│      │      └─ InventorySlots.cs
│      │      └─ InventorySlot.cs
│      │      └─Options.cs
│      │      └─ UsageOption.cs
```

```
│   │
│   ├─ Items/
│   │   └─ Item.cs
│   │
│   └─ ActionPanel.cs
│   └─ AttackSelector.cs
│   └─ BattleHP.cs
│   └─ BattleNPC.cs
│   └─ BattleOption.cs
│   └─ BattleUIManager.cs
│   └─ DialogSystem.cs
│   └─ InkControl.cs
│   └─ Player.cs
│   └─ PlayerMovement.cs
│   └─ QuickInfo.cs
│   └─ StateManager.cs
```

```
│       └─ Stats.cs
│       └─ UIRoot.cs
│       └─ EnumClasses.cs
└─ Ink/
```

### Pliki

## System ekwipunku (Inventory)

**System ekwipunku odpowiada za:**

- przechowywanie przedmiotów,
- zarządzanie uzbrojeniem (broń, pancerz),
- obsługę użycia przedmiotów,
- komunikację z UI poprzez eventy.

---

### Inventory.cs

**Główna klasa logiki ekwipunku.**

Pola:

- **List<Item> Items** - lista przedmiotów w ekwipunku

- **Item Weapon** - aktualnie założona broń
- **Item Armor** - aktualnie założony pancerz
- **int Capacity** - maksymalna liczba przedmiotów

---

## Kluczowe Eventy

**System oparty jest na eventach - UI aktualizuje się automatycznie:**

- **OnInventoryChanged(int)**
  wywoływany po dodaniu/usunięciu itemu
- **OnWeaponChanged(string, int)**
  nazwa + damage
- **OnArmorChanged(string, int)**
  nazwa + defense
- **OnItemUse(Item)**
  użycie przedmiotu

---

## Metody

### AddItem(Item item)

**Dodaje przedmiot do ekwipunku:**

- sprawdza null
- sprawdza capacity
- dodaje do listy
- wywołuje event

---

### RemoveItem(Item item, int index)

**Usuwa przedmiot:**

- po indeksie lub po referencji
- aktualizuje UI przez event

---

### SetWeapon(Item weapon)

**Ustawia broń:**

- sprawdza typ **(Weapon)**
- usuwa item z inventory
- wywołuje event UI

---

### SetArmor(Item armor)

**Analogicznie do broni:**

- sprawdza typ **(Armor)**
- aktualizuje UI

---

### WAŻNE:

### InventoryManagerUI.cs

Zarządza całym UI ekwipunku i menu.

### Enum Menu

Określa aktualny stan UI:

- Options
- Inventory
- ItemOptions
- Stats
- DialogueOptions

- MercyOptions
- Fight

### Główne elementy UI

- _Inventory – panel ekwipunku
- _QuickInfo – szybkie statystyki
- _Stats – pełne statystyki
- _Options – menu główne

---

## Kluczowe metody

### ProgressUI()

### Obsługa ENTER:

- Options  Inventory / Stats
- Inventory  ItemOptions
- ItemOptions  akcja (Use / Info / Drop)

---

### GetBackOrClose()

### Obsługa SHIFT:

- cofa UI o poziom wyżej
- zamyka menu

---

### CloseInventoryUI()

### Reset całego UI:

- wyłącza wszystkie panele
- resetuje indeksy

**Funkcjonalność:**

- dynamiczne tworzenie slotów
- aktualizacja po zmianie inventory
- nawigacja (góra/dół/lewo/prawo)

**Kluczowe metody:**

**UpdateSlots(int count)**

- czyści stare sloty
- tworzy nowe
- przypisuje nazwy itemów

---

**Activate(int index)**

- zaznacza slot
- zmienia aktualny index

---

**Move(Direction dir)**

**Obsługa nawigacji:**

- siatka 2 kolumny

---

## InventorySlot.cs

Reprezentuje pojedynczy slot UI.

**Funkcje:**

- SetItemName() – ustawia tekst
- Active() – pokazuje/ukrywa zaznaczenie

## Options.cs

Prosty system wyboru w menu.

---

## UsageOptions.cs

**Menu opcji dla itemów (Use / Info / Drop)**

### Funkcjonalność:

- poruszanie się lewo/prawo
- wybór opcji
- aktywacja UI

### Metody:

- **Move()** – zmiana opcji
- **Select()** – wybór
- **Activate()** – ustawienie aktywnej opcji

---

## Item.cs

**ScriptableObject definiujący pojedynczy przedmiot w grze (item).**

### Funkcjonalność:

- przechowywanie danych o przedmiocie
- obsługa różnych typów itemów (broń, leczenie, zbroja)
- możliwość użycia w systemie inventory / walki / ekwipunku
- definiowanie statystyk wpływających na gracza lub świat

### Kluczowe pola:

- **Name** – nazwa przedmiotu
- **Description** – opis przedmiotu
- **Value** – wartość (np. cena / punkty / gold)
- **ItemType** – typ przedmiotu (Default, Healing, Weapon, Armor)
- **HealingAmount – ilość** leczenia (dla itemów typu Healing)
- **NumberOfUses** – liczba użyć przedmiotu
- **Protection** – poziom obrony (dla armor)
- **Damage** – obrażenia (dla weapon)

**Konstruktor:**

**Item(string name, string description, int value, TypeOfItem itemType, int healingAmount, int numberOfUses, int protection, int damage)**

- pozwala zainicjalizować wszystkie dane itemu
- umożliwia tworzenie przedmiotów programowo (choć w ScriptableObject zwykle używa się edytora Unity)
- ustawia domyślne wartości dla parametrów opcjonalnych

**Enum: TypeOfItem**

**Określa kategorię przedmiotu:**

- **Default** – zwykły przedmiot
- **Healing** – przedmiot leczący
- **Weapon** – broń
- **Armor** – pancerz

## ActionPanel.cs

**Zarządza dynamicznym skalowaniem panelu UI (Action Panel) wraz z opcjonalną animacją zmiany rozmiaru.**

**Funkcjonalność:**

- dynamiczna zmiana rozmiaru panelu UI (RectTransform)
- obsługa presetów rozmiaru (Max / Square)
- możliwość ustawienia własnych wymiarów
- animowane przejścia między rozmiarami

- sterowanie szerokością i wysokością niezależnie

**Kluczowe pola:**

- **_Size –** referencja do RectTransform panelu
- **MaxWidth** – maksymalna szerokość panelu
- **SquareWidth** – szerokość trybu kwadratowego
- **DefaultHeight** – domyślna wysokość panelu
- **AnimationSpeed** – czas trwania animacji zmiany rozmiaru

**Główna metoda: SetSize(...)**

**Ustawia rozmiar panelu (z animacją lub bez).**

**Parametry:**

- **width** – docelowa szerokość (opcjonalna)
- **height** – docelowa wysokość (opcjonalna)
- **type** – preset rozmiaru:
  - **Max** - maksymalny rozmiar
  - **Square** - kwadratowy layout
- **animate** – czy używać animacji

**Logika:**

- jeśli **animate = true** - uruchamia coroutine **AnimateSize**
- jeśli **animate = false** - ustawia rozmiar natychmiast
- jeśli podano oba wymiary - używa ich bezpośrednio
- jeśli nie - używa presetów **(MaxWidth, SquareWidth)**

**Kluczowa metoda: AnimateSize(...)**

**Coroutine odpowiedzialna za płynną zmianę rozmiaru panelu.**

**Działanie:**

- zapisuje aktualny rozmiar **(startWidth, startHeight)**
- animuje wysokość przez **Mathf.Lerp**
- następnie animuje szerokość

- aktualizuje **RectTransform** co klatkę **(yield return null)**
- na końcu ustawia dokładne wartości docelowe

**Efekt:**

- płynne “rozciąganie” panelu UI
- osobna animacja osi pionowej i poziomej

**Enum: ActionPanelSize**

Określa preset rozmiaru panelu:

- **Square** – tryb kompaktowy / kwadratowy
- **Max** – pełny rozmiar panelu

**Enum: AnimationAxis**

**Określa oś animacji (obecnie informacyjnie):**

- **Horizontal** – szerokość
- **Vertical** – wysokość

**Zastosowanie w projekcie:**

- system walki (np. menu akcji jak w Undertale)
- dynamiczne UI zależne od stanu gry
- rozwijane panele wyboru
- animowane menu kontekstowe
- UX poprawiający czytelność interfejsu

---

**BattleHP.cs**

**Zarządza wyświetlaniem oraz dynamicznym skalowaniem paska HP w UI podczas walki.**

**Funkcjonalność:**

- wyświetlanie aktualnego HP w formacie current/max
- dynamiczne skalowanie paska HP względem wartości bazowej

- wizualizacja HP jako procentu (wypełnienie paska)
- automatyczne dopasowanie szerokości UI do maksymalnego HP

**Kluczowe pola:**

- **_HP** – tekst UI **(TMP_Text)** pokazujący HP
- **_HPLvl** – wewnętrzny pasek wypełnienia HP
- **_HPSize** – kontener całego paska HP
- **DefaultHPLvl** – referencyjna wartość HP do skalowania UI

**Zastosowanie:**

- system walki (HUD gracza i przeciwników)
- boss health bar
- dynamiczne UI zależne od statystyk postaci
- klasyczny pasek HP w stylu RPG / Undertale-like systemów

---

**BattleNPC.cs**

**Zarządza podstawową konfiguracją NPC w systemie walki, głównie elementami wizualnymi i identyfikacją.**

**Funkcjonalność:**

- przechowywanie i przypisywanie danych NPC w walce
- zarządzanie sprite'em NPC (postać)
- zarządzanie tłem NPC w UI
- identyfikacja NPC przez **`npcID`**
- przygotowanie NPC do systemu dialogów i UI walki

**Kluczowe pola:**

- **`_NPCDialogue`** – referencja do obiektu dialogowego NPC
- **`_NPCBackground`** – obraz tła NPC w UI
- **`_NPCSprite`** – sprite postaci NPC

- **`npcID`** – unikalny identyfikator NPC (np. do systemu dialogów lub logiki gry)

**Metoda: SetNPCsprite(Image sprite)**

**Ustawia grafikę NPC.**

**Działanie:**

- sprawdza czy przekazany sprite nie jest **`null`**
- przypisuje go do **`_NPCSprite`**

**Metoda: SetNPCBackground(Image sprite)**

**Ustawia tło NPC.**

**Działanie:**

- sprawdza czy przekazany sprite nie jest **`null`**
- przypisuje go do **`_NPCBackground`**

**Zastosowanie:**

- **s**ystem walki (NPC jako przeciwnik lub postać fabularna)
- dynamiczne zmiany wyglądu NPC w trakcie gry
- system dialogów **(po `_NPCDialogue`)**
- identyfikacja NPC w systemach questów i eventów **(`npcID`)**

**Efekt działania:**

- NPC może mieć dynamicznie zmieniany wygląd w UI
- łatwa integracja z systemem dialogowym
- prosta identyfikacja NPC w logice gry
- przygotowanie pod rozbudowany system walki / fabuły

---

**BattleOption.cs**

**Zarządza pojedynczą opcją w menu walki (np. Attack, Act, Item, Mercy), w tym**

**jej stanem aktywnym i wizualnym podświetleniem.**

### Funkcjonalność:

- zmiana wyglądu opcji UI (aktywny / nieaktywny stan)
- sterowanie widocznością wskaźnika (Heart)
- obsługa prostego highlightu w menu walki
- przygotowanie pod system wyboru akcji w stylu RPG

### Kluczowe pola:

- `ActiveSprite` – grafika opcji w stanie aktywnym
- `UnactiveSprite` – grafika opcji w stanie nieaktywnym
- `Heart` – wskaźnik (np. kursor wyboru)
- `Sprite` – komponent `Image` reprezentujący UI opcji

### Metoda: Start()

### Debug.Log("MENU START");

- log informacyjny potwierdzający inicjalizację elementu menu

### Metoda: Active(bool action)

Ustawia stan aktywności opcji w menu walki.

### Główna logika:

- jeśli `action == true:`
  - ustawia `ActiveSprite`
  - pokazuje `Heart`
- jeśli `action == false:`
  - ustawia `UnactiveSprite`
  - ukrywa `Heart`

### Efekt działania:

- opcje menu wizualnie reagują na wybór gracza
- aktywna opcja jest podświetlana
- wskaźnik (Heart) pokazuje aktualnie wybraną akcję
- UI przypomina system wyboru akcji w stylu Undertale

**Zastosowanie:**

- system walki (menu akcji gracza)
- nawigacja po opcjach (Attack / Act / Item / Mercy)
- wizualne feedbacki UI
- integracja z systemem sterowania inputem

---

### BattleOptions.cs

**Zarządza zestawem opcji w menu walki (np. Attack / Act / Item / Mercy), w tym nawigacją, aktywacją i wyborem.**

**Funkcjonalność:**

- przechowywanie listy opcji menu
- sterowanie aktualnie zaznaczoną opcją
- nawigacja lewo/prawo
- aktywacja i dezaktywacja opcji UI
- wybór aktualnie zaznaczonej akcji
- reset menu do stanu początkowego

**Kluczowe pola:**

- `CurrIndex` – aktualny indeks zaznaczonej opcji
- `Options` – lista dostępnych opcji **(`BattleOption`)**

**Metoda: Off()**

Wyłącza wszystkie opcje menu.

**Działanie:**

- przechodzi przez wszystkie opcje
- ustawia je jako nieaktywne **`(Active(false))`**
- resetuje **`CurrIndex`** do **`0`**

### Metoda: Move(Direction dir)

**Obsługuje nawigację po menu.**

### Działanie:

- **`Left`** - przejście do poprzedniej opcji
- **`Right`** - przejście do następnej opcji
- wywołuje **`Activate()`** z nowym indeksem

### Metoda: Select()

**Zwraca aktualnie wybraną opcję.**

### Działanie:

- sprawdza czy **`CurrIndex`** jest w zakresie listy
- dezaktywuje aktualną opcję
- zwraca indeks wybranej opcji
- jeśli coś jest nie tak - zwraca **`-1`**

### Metoda: Activate(int index)

**Aktywuje konkretną opcję w menu.**

### Logika:

- sprawdza czy index jest poprawny
- wyłącza poprzednią opcję
- włącza nową opcję
- aktualizuje **`CurrIndex`**
- zwraca **`true`** jeśli sukces, **`false`** jeśli błąd

### Efekt działania:

- menu walki działa jak klasyczny selector (Undertale-like)

- tylko jedna opcja jest aktywna na raz
- gracz może poruszać się między opcjami
- wybór opcji zwraca indeks do dalszej logiki walki

### Zastosowanie:

- system walki RPG
- menu akcji gracza
- sterowanie inputem (klawiatura / pad)
- integracja z systemem dialogów lub Ink
- baza pod bardziej rozbudowane drzewo akcji

## BattleUIManager.cs

**Główny kontroler systemu UI walki. Zarządza stanami menu, nawigacją, przejściami między scenami UI oraz integracją z systemami gracza, NPC, inventory i dialogów.**

### Funkcjonalność:

- centralne sterowanie UI walki (menu stanów)
- obsługa nawigacji (kierunki: góra/dół/lewo/prawo)
- przełączanie między ekranami (Options, Fight, Inventory, Dialogue itd.)
- integracja z:
  - systemem dialogów (Ink)
  - inventory gracza
- zarządzanie flow walki (wybór akcji - podmenu - wykonanie akcji)
- cofanie się w UI (back system)

### Enum: Direction

**Kierunki nawigacji UI:**

- Up
- Down

- Left
- Right

### Kluczowe pola:

**UI i sceny:**

- `_BattleUI` – główny UI walki
- `_Panel` – panel akcji
- `_FightScene` – scena ataku
- `_AttackScene` – UI ataku
- `_DialogueScene` – UI dialogów
- `_Inventory` – UI inventory
- `_ItemDescription` – opis itemu

### Systemy gry:

- `battleNPC` – NPC w walce
- `battleHP` – HUD HP
- `battleOptions` – menu główne
- `inventorySlots` – sloty inventory
- `usageOptions` – opcje użycia itemu
- `player` – gracz
- `inkController` – system dialogów Ink

### Stan:

- `SelectedItem` – aktualnie wybrany item
- `CurrentMenu` – aktualny stan UI (FSM)

### Stałe:

- `OptionsCount` – liczba opcji głównych

- `itemOptionsCount` – liczba opcji itemów

**Start()**

- **ustawia domyślną aktywną opcję w menu (`Activate(0)`)**

**OnEnable / OnDisable**
**Subskrypcje eventów:**

- `Player.OnPlayerHpChanged` - aktualizacja HP UI
- `player.inventory.OnInventoryChanged` - aktualizacja slotów inventory

**Update() – Input system**
Obsługuje sterowanie:

**Strzałki:**

- wywołują `ChooseOptionOrItem()`

**Enter:**

- jeśli dialog aktywny - `ShowNextLine()`
- jeśli nie - `ProgressUI()`

**RightShift:**

- cofanie UI (`GetBack()`)

**ChooseOptionOrItem(Direction)**
**Obsługuje nawigację zależnie od aktualnego menu:**

- `Options` - battleOptions
- `Inventory` - inventorySlots
- `ItemOptions` - usageOptions
- jeśli nie jesteśmy w `BattleMenu` - ignoruje input

## ProgressUI() – główna logika wyboru

### Menu: Options

- Attack - FightScene
- Act - Dialogue (Ink "act")
- Item - Inventory
- Mercy - Dialogue (Ink "mercy")

### Menu: Inventory

- wybór itemu - przejście do ItemOptions

### Menu: ItemOptions

- Use - użycie itemu + powrót
- Back - powrót

## GetBack() – system cofania UI

### ItemOptions - Inventory

- zamyka opis itemu
- resetuje usage options
- przywraca wybór itemu

### Inventory - Options

- zamyka inventory
- resetuje sloty
- wraca do menu głównego

### Efekt działania:

- pełny system walki w stylu Undertale-like FSM
- płynne przejścia między stanami UI

- logiczny flow:
  Options - submenus - action - return
- integracja dialogów, inventory i walki w jednym kontrolerze

**Zastosowanie:**

- core system walki RPG
- sterowanie całym UI gry
- integracja gameplay + dialog + item system
- fundament pod rozbudowę walki (animacje, enemy AI, efekty)

**DialogSystem.cs**

**Prosty system dialogów UI odpowiedzialny za wyświetlanie i ukrywanie panelu dialogowego oraz tekstu.**

**Funkcjonalność:**

- otwieranie panelu dialogowego
- wyświetlanie tekstu dialogu
- zamykanie dialogu
- testowe sterowanie klawiszem **(E)**
- kontrola stanu dialogu **(`isOpen`)**

**Kluczowe pola:**

- **`dialogPanel`** – obiekt UI panelu dialogowego
- **`dialogText`** – tekst dialogu **(`TMP_Text`)**
- **`isOpen`** – informacja czy dialog jest aktualnie aktywny

**Update() – testowe sterowanie:**

**Klawisz E:**

- jeśli dialog jest otwarty - zamyka go
- jeśli zamknięty - otwiera testowy dialog `"przykladowy dialog"`

**Metoda: OpenDialog(string text)**

**Otwiera dialog i ustawia jego treść.**

**Działanie:**

- aktywuje panel UI
- ustawia tekst dialogu
- zmienia stan `isOpen` na `true`

**Metoda: CloseDialog()**

**Zamyka dialog.**

**Działanie:**

- **ukrywa panel UI**
- **ustawia `isOpen` na `false`**

**Efekt działania:**

- prosty system popup dialogów
- szybkie testowanie UI dialogowego
- podstawowy kontroler tekstu w grze
- łatwa integracja z NPC / systemem walki

**Zastosowanie:**

- dialogi NPC
- tutoriale i komunikaty systemowe
- system fabularny

- integracja z bardziej zaawansowanym systemem (np. Ink)

### InkControll.cs

**Kontroler systemu dialogów oparty na Ink (Inkle), odpowiedzialny za uruchamianie, prowadzenie i kończenie dialogów w grze.**

### Funkcjonalność:

- uruchamianie dialogów z pliku Ink **(`TextAsset`)**
- wybór ścieżki dialogowej na podstawie NPC i akcji
- wyświetlanie kolejnych linii dialogu
- kontrola stanu aktywnego dialogu
- automatyczne kończenie dialogu i ukrywanie UI

### Kluczowe pola:

- **`inkJSON`** – plik dialogowy Ink **(`.json`** wygenerowany z Ink`)
- **`dialogueText` – UI tekst (`TextMeshProUGUI`)**
- **`story`** – aktualna instancja Ink Story
- **`dialogueActive`** – informacja czy dialog jest aktywny

### Metoda: StartDialogue()

**Uruchamia dialog dla konkretnego NPC i akcji.**

### Działanie:

- tworzy nową instancję **`Story`**
- wybiera ścieżkę w Ink
- aktywuje dialog UI

- uruchamia pierwszą linię

### Metoda: IsDialogueActive()

Zwraca czy dialog jest aktualnie aktywny.

### Używane do:

- blokowania inputu w UI walki
- sterowania przejściem dialogów

### Efekt działania:

- dynamiczny system dialogów oparty na Ink
- dialogi zależne od NPC i akcji
- płynne przechodzenie linia po linii
- automatyczne zamykanie po zakończeniu historii
- integracja z systemem walki i UI

### Zastosowanie:

- system ACT / MERCY w walce
- dialogi NPC w grze
- narracja fabularna
- eventy i cutscenki
- rozbudowane drzewka dialogowe Ink

## AttackSelector.cs

Kontroler systemu wyboru ataku w mini-grze, odpowiedzialny za poruszanie wskaźnika i wybór wartości trafienia.

### Funkcjonalność:

- przesuwanie wskaźnika ataku
- wykrywanie momentu wyboru
- obliczanie precyzji trafienia
- resetowanie pozycji po wyborze
- emitowanie wyniku przez event

### Kluczowe pola:

- `SelectorRigbody` – fizyka ruchu wskaźnika
- `SelectorRect` – pozycja UI
- `Speed` – prędkość ruchu wskaźnika
- `StartPosition` – początkowa odległość od środka

### Update() – logika wyboru

### OnDisable() - resetuje stan slidera

**Metoda: SetOff()**

**Resetuje wskaźnik do pozycji startowej.**

**Metoda: Select()**

**Finalizuje wybór ataku.**

## EnumClasses.cs (System Enumów UI i gry)

Zestaw podstawowych enumów wykorzystywanych w systemie walki, UI oraz stanach gry.

### Direction

Określa kierunek nawigacji w UI:

- `Up` – góra
- `Down` – dół
- `Right` – prawo
- `Left` – lewo

### Menu

Definiuje aktualny stan menu w systemie walki/UI.

- `Options` – główne menu akcji
- `Inventory` – ekran ekwipunku
- `ItemOptions` – opcje użycia itemu
- `ItemDescriprion` – opis itemu
- `Stats` – statystyki
- `DialogueOptions` – dialogi
- `MercyOptions` – opcje mercy
- `Fight` – faza walki / ataku

### GameState

- `Gameplay` – normalna gra
- `MainMenu` – menu główne
- `BattleMenu` – menu walki
- `GameplayDialogue` – dialogi poza walką
- `BattleDialogue` – dialogi w walce
- `Fight` – aktywna walka / faza ataku
- `CutScene` – sceny przerywnikowe

### ActionPanelSize

Rozmiar panelu akcji UI:

- `Square` – kompaktowy / kwadratowy
- `Max` – pełny rozmiar panelu

**Użycie:**

- **ActionPanel.cs**

### AnimationAxis

Oś animacji UI:

- `Horizontal` – szerokość
- `Vertical` – wysokość

**Użycie:**

- animacja UI paneli
- ActionPanel (rozciąganie)

**Efekt całości:**

- spójny system stanów gry i UI
- łatwa kontrola flow walki
- czytelna struktura nawigacji

- przygotowanie pod rozbudowę RPG / Undertale-like systemu

---

### ItemDescription.cs

**Prosty kontroler UI odpowiedzialny za wyświetlanie opisu aktualnie wybranego przedmiotu.**

### Funkcjonalność:

- wyświetlanie tekstu opisu itemu w UI
- aktualizacja treści dynamicznie podczas wyboru przedmiotów
- prosta integracja z systemem inventory i menu walki

### Kluczowe pola:

- `_ItemDescription` – komponent tekstowy (TMP_Text) wyświetlający opis przedmiotu

### Metoda: SetDescription(string text)

**Ustawia opis aktualnie wybranego itemu.**

### Działanie:

- przypisuje przekazany tekst do UI:

### Efekt działania:

- UI automatycznie pokazuje opis wybranego przedmiotu
- gracz widzi szczegóły itemu w czasie wyboru
- prosty system informacji kontekstowej

### Zastosowanie:

- inventory system
- menu użycia itemów w walce

- podgląd statystyk przedmiotów

---

### NPC.cs

**Klasa reprezentująca przeciwnika lub postać NPC w świecie gry, z podstawową logiką walki, HP oraz eventami interakcji z graczem.**

#### Funkcjonalność:

- przechowywanie danych NPC
- obsługa obrażeń i śmierci NPC
- wykrywanie kontaktu z graczem
- wywoływanie eventów przy spotkaniu NPC
- przygotowanie NPC do systemu walki

#### Kluczowe pola:

- `npcName` – nazwa NPC
- `maxHP` – maksymalne HP
- `currentHP` – aktualne HP
- `canFight` – czy NPC może wejść w walkę
- `battleSprite` – sprite używany w UI walki

#### Eventy:

- `OnPlayerMeetNPC (NPC)` – wywoływany gdy gracz wejdzie w trigger NPC
- `OnNPCDeath (NPC)` – (zadeklarowany, ale aktualnie nieużywany w kodzie)

---

**Start()**

- ustawia `currentHP = maxHP`

**Metoda: TakeDamage(int dmg)**

Obsługa otrzymywania obrażeń.

**Działanie:**

- odejmuje HP

**OnTriggerEnter2D(Collider2D other)**

Wykrywanie kontaktu z graczem.

**Działanie:**

- loguje spotkanie
- wywołuje event:

**Efekt działania:**

- NPC może inicjować walkę po kontakcie z graczem
- system oparty na eventach (łatwa integracja z UI / battle system)
- prosta logika HP i eliminacji NPC
- przygotowanie pod system RPG encounters

**Zastosowanie:**

- system walki w stylu RPG
- overworld encounters
- bossy i przeciwnicy

- event-driven rozpoczęcie **BattleUIManager**
- integracja z systemem dialogów i Ink

---

### Player.cs

Główna klasa gracza odpowiedzialna za statystyki, ekwipunek, kalkulację obrażeń oraz komunikację z systemem UI poprzez eventy.

### Funkcjonalność:

- przechowywanie statystyk gracza
- integracja z systemem inventory
- obliczanie ataku i obrony na podstawie ekwipunku
- leczenie gracza i aktualizacja HP UI
- wywoływanie eventów do UI
- wykonywanie użycia itemów

### Kluczowe pola:

- **`inventory`** – ekwipunek gracza
- **`LVL`** – poziom postaci
- **`EXP`** – doświadczenie
- **`GOLD`** – waluta
- **`BaseHP`** – maksymalne HP
- **`HP`** – aktualne HP
- **`BaseDamage`** – podstawowe obrażenia

### Eventy:

- **`OnPlayerLvlChanged`** – zmiana poziomu
- **`OnPlayerGoldChanged`** – zmiana złota
- **`OnPlayerBaseHpChanged`** – zmiana maks HP *(zadeklarowane, ale nieużywane w kodzie)*

- `OnPlayerHpChanged` – zmiana HP (current, base)

**Użycie:**

- UI (HP bar, HUD)
- system walki
- inventory UI
- statystyki gracza

**Start()**

**Wysyła początkowe wartości do UI:**

- LVL
- GOLD
- HP / BaseHP

**Metoda: GetAtack()**

Oblicza obrażenia gracza.

**Metoda: GetDefence()**

Oblicza obronę gracza.

**Logika:**

- jeśli armor istnieje - zwraca `Protection`
- jeśli nie - 0

**Metoda: UseItem(int index)**

- deleguje użycie itemu do inventory
- przekazuje referencję do gracza (np. leczenie)

**Metoda: Heal(int amount)**

Leczy gracza.

**Działanie:**

- nie przekracza `BaseHP`

- aktualizuje HP
- wywołuje event:

**Metoda: InventorySize()**

Zwraca ilość itemów w ekwipunku.

**Efekt działania:**

- centralna klasa gracza w systemie RPG
- dynamiczne statystyki zależne od ekwipunku
- pełna integracja z UI przez eventy
- obsługa walki, leczenia i itemów

**Zastosowanie:**

- system walki (damage / defence)
- HUD (HP, LVL, GOLD)
- inventory system
- RPG progression system
- integracja z BattleUIManager i NPC

---

**PlayerMovement.cs**

**Sterownik ruchu gracza (oraz opcjonalnie “heart” w walce), oparty na Rigidbody2D i systemie Input System.**

**Funkcjonalność:**

- obsługa ruchu gracza w oparciu o Input System **(`Vector2`)**
- fizyczne poruszanie obiektu przez **`Rigidbody2D`**
- rozdzielenie ruchu:
  - normalna gra **(`Gameplay`)**
  - tryb walki / heart **(`Fight`)**
- blokowanie ruchu w zależności od stanu gry

**Kluczowe pola:**

- `movementSpeed` – prędkość ruchu
- `isFightHeart` – czy obiekt jest “sercem” w walce (Undertale-like movement)
- `myRigidbody` – komponent fizyki
- `moveInput` – aktualny kierunek ruchu

### Update() – kontrola stanu gry

**Działanie:**

- pobiera wektor ruchu
- zapisuje do `moveInput`

**Efekt działania:**

- prosty i stabilny movement system
- działa zarówno dla gracza, jak i systemu walki
- sterowanie zależne od globalnego stanu gry

**Zastosowanie:**

- movement w świecie gry
- system walki
- input system Unity

---

### QuickInfo.cs

**Prosty kontroler UI odpowiedzialny za szybkie wyświetlanie podstawowych statystyk gracza w interfejsie.**

**Funkcjonalność:**

- wyświetlanie poziomu gracza
- wyświetlanie HP
- wyświetlanie ilości złota
- szybka aktualizacja HUD bez dodatkowej logiki

---

**Kluczowe pola:**

- `_LVL` – tekst poziomu gracza
- `_HP` – tekst HP
- `_GOLD` – tekst złota

**Metoda: SetLvl()**

**Ustawia poziom gracza:**

- konwertuje int - string
- aktualizuje UI

**Metoda: SetHp()**

**Ustawia HP w formacie:**

**hp/baseHp**

- pokazuje aktualne i maksymalne HP

**Metoda: SetGold()**

**Ustawia ilość złota:**

- konwersja liczby na tekst
- aktualizacja UI

**Efekt działania:**

- prosty HUD gracza
- szybka aktualizacja statystyk
- czytelne informacje podczas gameplayu i walki
- brak logiki - tylko prezentacja danych

**Zastosowanie:**

- UI gry (HUD)
- system walki

- inventory
- integracja z eventami **Player.cs**

---

### StateManager.cs

**Prosty globalny kontroler stanu gry (GameState), odpowiedzialny za przechowywanie aktualnego trybu rozgrywki.**

#### Kluczowe pole:

- **`CurrentGameState`** – statyczny aktualny stan gry **(`GameState`)**

#### Dzięki `static`:

- dostępny globalnie bez referencji do obiektu
- używany w UI, movement, combat i dialogach

#### Efekt działania:

- centralny punkt kontroli stanu gry
- inne systemy mogą blokować/uruchamiać logikę w zależności od stanu
- prosta implementacja FSM na poziomie globalnym

#### Zastosowanie:

- BattleUIManager
- PlayerMovement
- dialog system
- system walki i cutscen

---

### Stats.cs

**Kontroler UI odpowiedzialny za wyświetlanie pełnych statystyk gracza w panelu informacji.**

#### Funkcjonalność:

- wyświetlanie podstawowych statystyk gracza
- prezentacja statystyk walki
- pokaż aktualnie wyposażonej broni i pancerza
- aktualizacja UI w czasie rzeczywistym
- warstwa tylko do prezentacji danych

**Kluczowe pola:**

- **`_LVL`** – poziom gracza
- **`_HP`** – aktualne HP / maksymalne HP
- **`_GOLD`** – ilość złota
- **`_ATACK`** – obrażenia gracza
- **`_DEFENCE`** – obrona gracza
- **`_WEAPON`** – nazwa broni
- **`_ARMOR`** – nazwa pancerza

**Metody:**

**SetLvl**

- aktualizuje poziom gracza w UI

**SetHp**

- wyświetla HP w formacie:

**hp/baseHp**

**SetGold**

- aktualizuje ilość złota

**SetAtack**

- wyświetla aktualny atak gracza

---

**SetDefence**

- wyświetla obronę

**SetWeapon**

- pokazuje nazwę aktualnie wyposażonej broni

**SetArmor**

- pokazuje nazwę pancerza

**Efekt działania:**

- pełny panel statystyk RPG
- czytelny podgląd postaci
- rozdzielenie logiki gry od UI
- łatwa integracja z eventami **Player.cs**

**Zastosowanie:**

- menu statystyk
- ekwipunek i wyposażenie
- HUD rozszerzony
- debug / development overlay

**UIRoot.cs**

Prosty kontroler obiektu UI, który zapewnia jego trwałość pomiędzy scenami.

**Funkcjonalność:**

- utrzymuje UI aktywne po zmianie sceny
- zapobiega niszczeniu obiektu przy ładowaniu nowej sceny

- tworzy globalny “root” dla całego interfejsu użytkownika

**Kluczowa metoda: Awake()**

**Działanie:**

- wywoływana przed `Start()`
- zabezpiecza obiekt przed zniszczeniem:

**Efekt działania:**

- UI nie resetuje się przy zmianie sceny
- HUD, menu i battle UI pozostają aktywne
- możliwa ciągła kontrola interfejsu w całej grze

**Zastosowanie:**

- systemy UI w grach wieloscenowych
- battle UI i inventory
- globalne menu gry